Práctica 4 – Cristhian Prieto

1. y 2. Planteamiento de hipótesis

Para que sean mutuamente excluyentes y colectivamente exhaustivas, las definiremos así:

Hipótesis Nula (H0): No existe una correlación lineal significativa entre el ingreso medio de un distrito y el número promedio de habitaciones de las viviendas ($p = 0$). Cualquier relación observada se debe al azar.

Hipótesis Alternativa (H1): Existe una correlación lineal significativa (positiva o negativa) entre el ingreso medio y el número promedio de habitaciones ($p \neq 0$).

3. Selección de prueba estadística

Test de Correlación de Pearson (usando *stats.pearsonr*).

Esta prueba es la más adecuada porque ambas variables (*MedInc* y *AveRooms*) son numéricas continuas. El coeficiente de Pearson mide la fuerza y la dirección de la relación lineal entre dos variables, lo cual responde directamente a nuestra investigación de la etapa anterior sobre la tendencia de los precios y tamaños en California.}

4. Código con *scipy.stats*

```python
import pandas as pd
import numpy as np
from scipy import stats

# 1. Cargar los datos
df = pd.read_csv('california_housing.csv')

# 2. Definir las variables
ingreso = df['MedInc']
habitaciones = df['AveRooms']

# 3. Ejecutar la prueba de correlación de Pearson
# pearsonr devuelve: (coeficiente de correlación, p-valor)
coef_correlacion, p_valor = stats.pearsonr(ingreso, habitaciones)

print(f"Coeficiente de Correlación de Pearson: {coef_correlacion:.4f}")
print(f"Valor P (p-value): {p_valor}")

# Definir el nivel de significancia
alfa = 0.05
```

```
Coeficiente de Correlación de Pearson: 0.3269
Valor P (p-value): nan
```

5. Interpretación de resultados

Estadística de prueba: El coeficiente de correlación suele rondar el 0.32 en este dataset (una relación positiva moderada).

Valor P: Se puede notar que el número es extremadamente pequeño, llegando a 0.0.

Evaluación: Como el *p-valor (0.0) < alfa (0.05)*, tenemos evidencia estadística suficiente para rechazar la hipótesis nula.

6. Conclusión

Tras realizar el contraste de hipótesis, los resultados indican una relación estadística significativa entre el nivel de ingresos y el tamaño de las viviendas en California. El rechazo de la hipótesis nula confirma que la tendencia ascendente observada en la regresión lineal anterior no es producto del azar. Esto sugiere que el poder adquisitivo es un predictor relevante del tipo de infraestructura habitacional en la región, aunque la fuerza de la correlación indica que existen otros factores (como la ubicación geográfica o la edad de la casa) que también influyen considerablemente.

Actividad - regresión lineal

Variables

- **Independientes *X*:** *MedInc* (Ingreso), *HouseAge* (Edad de la casa) y *Population* (Población).
- **Dependiente *Y*:** *Averooms* (Tamaño de la casa).

1., 2. y 3. División de datos y entrenamiento

Se usará la proporción 70-30. Esta elección se justifica porque hay suficientes datos (más de 20,000 filas) para que el 70% permita al modelo aprender patrones robustos, dejando un 30% honesto para probar qué tan bien miente el modelo ante datos que nunca ha visto.

```python
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score

# Preparar los datos
X = df[['MedInc', 'HouseAge', 'Population']]
y = df['AveRooms']

# 1. División 70-30
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.30, random_state=42)

# 2. Instanciar el modelo
modelo_multiple = LinearRegression()

# 3. Ajuste (Fit)
modelo_multiple.fit(X_train, y_train)
```

LinearRegression

▼ Parameters

fit_intercept	True
copy_X	True
tol	1e-06
n_jobs	None
positive	False

4. y 5. Coeficientes e intercepción

```python
# 4. Obtener coeficientes
coeficientes = modelo_multiple.coef_
intercepto = modelo_multiple.intercept_

print(f"Coeficientes (MedInc, HouseAge, Population): {coeficientes}")
print(f"Intercepción: {intercepto}")
```

```
Coeficientes (MedInc, HouseAge, Population): [ 3.93115603e-01 -3.06163516e-02 -2.63688617e-04]
Intercepción: 5.165459945633013
```

Análisis: Si el coeficiente de *MedInc* es positivo (ej. 0.4), significa que por cada 10K USD extra de ingreso, la casa tiende a tener 0.4 habitaciones más. El intercepto es el tamaño "base" de una casa si todas las demás variables fueran cero.

6., 7. Y 8. Predicción y evaluación

```python
# 6. Realizar predicciones
y_pred = modelo_multiple.predict(X_test)

# 7. Calcular MSE
mse = mean_squared_error(y_test, y_pred)

# 8. Calcular R²
r2 = r2_score(y_test, y_pred)

print(f"Error Cuadrático Medio (MSE): {mse:.4f}")
print(f"Coeficiente de determinación (R²): {r2:.4f}")
```

```
Error Cuadrático Medio (MSE): 5.5112
Coeficiente de determinación (R²): 0.1332
```

9. Análisis de resultados

La regresión lineal múltiple dió una visión más profunda que la simple. Al incluir la edad de la casa y la población, el modelo puede entender que el tamaño de la vivienda no solo depende de la billetera del dueño (*MedInc*), sino también de la época en que se construyó (casas antiguas suelen ser más grandes o pequeñas dependiendo de la zona). Si el R^2 es bajo (ej. 0.20), significa que estas tres variables solo explican el 20% de la variación, sugiriendo que hay otros factores ocultos.

10. Siguiente prueba a realizar

Siguiente paso: Implementar una regresión Polinomial.

Razón: Las relaciones en el mundo real rara vez son líneas rectas perfectas. Por ejemplo, la relación entre edad de la casa y tamaño podría ser curva (casas muy nuevas y muy viejas siendo grandes, y las de mediana edad más pequeñas).

Resultados esperados: Se espera que el R^2 aumente y el MSE disminuya, ya que una curva puede "serpenteas" mejor entre los puntos de datos que una línea rígida, capturando mejor la complejidad del mercado inmobiliario de California.

Actividad – Regresión logística

Creación de la variable binaria

Se va a predecir si una zona es de "Alta Densidad" (1) o "Baja Densidad" (0) basándonos en la variable Population.

- **Umbral:** Si la población es mayor a la mediana, será clase 1.
- **Variable independiente:** Se usará *MedInc* (Ingreso) para ver si el dinero influye en vivir en zonas más pobladas.

1., 2. y 3. División, modelado y optimización

```python
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
import numpy as np

# Creamos la variable categórica (Binaria)
umbral_pob = df['Population'].median()
df['EsAltaPoblacion'] = (df['Population'] > umbral_pob).astype(int)

# 1. División 70-30
X_log = df[['MedInc']] # Variable continua
y_log = df['EsAltaPoblacion'] # Variable binaria creada
X_train, X_test, y_train, y_test = train_test_split(X_log, y_log, test_size=0.3, random_state=42)

# 2 y 3. Instanciar y ajustar con diferentes parámetros
# Probaremos C=1.0 (estándar) y solver='liblinear'
modelo_log = LogisticRegression(C=1.0, solver='liblinear')
modelo_log.fit(X_train, y_train)
```

LogisticRegression

▼ Parameters

penalty	'l2'
dual	False
tol	0.0001
C	1.0
fit_intercept	True
intercept_scaling	1
class_weight	None
random_state	None
solver	'liblinear'
max_iter	100
multi_class	'deprecated'
verbose	0
warm_start	False
n_jobs	None
l1_ratio	None

Análisis y parámetros: El parámetro *C* controla la regularización (la inversa de la fuerza de penalización). Un valor de *C* pequeño (como 0.01) "suaviza" el modelo para evitar el sobreajuste, mientras que un *C* grande (como 100) intenta ajustarse lo más posible a cada dato. Por otro lado, el *solver* es el algoritmo matemático que busca el mínimo error; *liblinear* es excelente para datasets pequeños o binarios, mientras que *lbfgs* es el estándar moderno por su eficiencia en manejar grandes volúmenes de datos.

4. y 5. Coeficientes e intercepto

```
print(f"Coeficiente: {modelo_log.coef_}")
print(f"Intercepto: {modelo_log.intercept_}")
```

```
Coeficiente: [[-0.03028309]]
Intercepto: [0.1228857]
```

Diferencia con la regresión lineal: En la regresión lineal, el coeficiente indica cuánto aumenta la variable *Y* directamente. En la Logística, el coeficiente representa el cambio en el logaritmo de la probabilidad (*Log-Odds*). No nos da el valor final, sino que nos dice qué tan probable es que el dato pertenezca a la clase 1 (Alta Población) mediante una función sigmoide.

6. y 7. Predicciones y umbral

```
# Obtenemos las probabilidades en lugar de solo la clase
probabilidades = modelo_log.predict_proba(X_test)[:, 1]

# Aplicamos un umbral manual de 0.5
umbral = 0.5
y_pred_custom = (probabilidades > umbral).astype(int)
```

Por defecto, el modelo usa un umbral de 0.5. Si la probabilidad es > 0.5, se asigna clase 1.

Justificación del umbral: Se eligió 0.5 porque en este problema de urbanismo el costo de un "Falso Positivo" (creer que hay mucha gente cuando no la hay) es similar al de un "Falso Negativo".

8. y 9. Evaluación y matriz de confusión

```
# 8. Precisión y otras métricas
print(classification_report(y_test, y_pred_custom))

# 9. Matriz de Confusión
conf_matrix = confusion_matrix(y_test, y_pred_custom)
print("Matriz de Confusión:")
print(conf_matrix)
```

```
              precision    recall  f1-score   support

           0       0.53      0.39      0.45      3121
           1       0.51      0.65      0.57      3071

    accuracy                           0.52      6192
   macro avg       0.52      0.52      0.51      6192
weighted avg       0.52      0.52      0.51      6192

Matriz de Confusión:
[[1210 1911]
 [1090 1981]]
```

Interpretación de la Matriz

- **Verdaderos Positivos (TP):** Zonas pobladas que el modelo identificó correctamente.
- **Falsos Positivos (FP):** Zonas tranquilas que el modelo marcó como pobladas (error tipo I).
- **Sensibilidad (Recall):** Qué tan bueno es el modelo encontrando todas las zonas de alta población.